Architecture 1001: x86-64 Assembly Intel vs. AT&T Syntax

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

Intel vs. AT&T Syntax

Intel vs. AT&T Syntax

- My classes prefer Intel syntax

Intel vs. AT&T Syntax

- My classes prefer Intel syntax
- But I learned AT&T syntax first

Intel vs. AT&T Syntax

- My classes prefer Intel syntax
- But I learned AT&T syntax first
- I believe it's important to know both, so you can read documents in either format

Intel vs. AT&T Syntax

- My classes prefer Intel syntax
- But I learned AT&T syntax first
- I believe it's important to know both, so you can read documents in either format
- Origin story: Bell Labs -> Multics on mainframes -> UNIX on PDP-11 *asm* -> UNIX on PDP-11 *C* -> UNIX on 8086 *C* (with assembly syntax adapted from PDP-11) -> GNU's Not Unix (╯°□°)╯ ┻━┻

https://stackoverflow.com/questions/42244028/what-was-the-original-reason-for-the-design-of-att-assembly-syntax
http://www.columbia.edu/~hauben/book-pdf/CHAPTER%209.pdf
https://en.wikipedia.org/wiki/History_of_Unix
https://www.bell-labs.com/usr/dmr/www/otherports/newp.pdf

Intel vs. AT&T Syntax 2

- Intel Syntax:
  - Preferred on Windows. Think algebra or C: y = 2x + 1;
  - Destination <- Source(s) *(right to left)*
  - mov rbp, rsp
  - add rsp, 0x14 ; (rsp = rsp + 0x14)
- AT&T Syntax:
  - Preferred on *nix/GNU. Think elementary school: 1 + 2 = 3
  - Source(s) -> Destination (left to right)
  - mov %rsp, %rbp
  - add $0x14,%rsp
  - Registers get a % prefix and immediates get a $

https://sourceware.org/binutils/docs/as/i386_002dVariations.html#i386_002dVariations

Intel vs. AT&T Syntax 3

- Intel indicates size with things like “mov *qword ptr* [rax], rbx", but it’s not in the actual mnemonic of the instruction
- "In AT&T syntax the size of memory operands is determined from the last character of the instruction mnemonic. ...only when there’s no other way to disambiguate an instruction"
  - mov**b** - operates on bytes
  - mov**w** - operates on word (2 bytes)
  - mov**l** - operates on “long” (dword) (4 bytes)
  - mov**q** - operates on “quad word” (qword) (8 bytes)

https://sourceware.org/binutils/docs/as/i386_002dVariations.html#i386_002dVariations

Intel vs. AT&T Syntax 4

- Some mnemonics have been more or less renamed, to better conform to the b/w/l/q mnemonic naming conventions for lengths
  - "cwde" -> "cwtl" ("**c**onvert (sign extend) **w**ord **t**o **l**ong")
  - "movsx" -> "movsbw"
- **I'm not a big fan of this!** It makes it *way harder* to look up instructions in the Intel manual when you're first learning assembly! (Basically you're forced to google for your solutions)

https://sourceware.org/binutils/docs/as/i386_002dMnemonics.html#Instruction-Naming

Intel vs. AT&T Syntax 5

```
endbr64
sub    $0x1000,%rsp
orq    $0x0,(%rsp)
sub    $0x1000,%rsp
orq    $0x0,(%rsp)
sub    $0x28,%rsp
movl   $0xff,0x1010(%rsp)
mov    0x8(%rsi),%rdi
mov    $0xa,%edx
mov    $0x0,%esi
callq  0x555555555070 <strtol@plt>
movslq %eax,%rdx
lea    0x1010(%rsp),%rsi
mov    %rsp,%rdi
mov    $0x1004,%ecx
callq  0x555555555060 <__memcpy_chk@plt>
movabs $0xace0fba5e,%rax
add    $0x2028,%rsp
retq
```

```
endbr64
sub    rsp,0x1000
or     QWORD PTR [rsp],0x0
sub    rsp,0x1000
or     QWORD PTR [rsp],0x0
sub    rsp,0x28
mov    DWORD PTR [rsp+0x1010],0xff
mov    rdi,QWORD PTR [rsi+0x8]
mov    edx,0xa
mov    esi,0x0
call   0x555555555070 <strtol@plt>
movsxd rdx,eax
lea    rsi,[rsp+0x1010]
mov    rdi,rsp
mov    ecx,0x1004
call   0x555555555060 <__memcpy_chk@plt>
movabs rax,0xace0fba5e
add    rsp,0x2028
ret
```

Intel vs. AT&T Syntax 5

```
movslq %eax,%rdx
```

```
movsxd rdx,eax
```

Intel vs. AT&T Syntax 6

- In my opinion the hardest-to-read difference is for r/mX values
- For intel it's expressed as
  `[base + index*scale + disp]`
- For AT&T it's expressed as
  `disp(base, index, scale)`
- So basically it makes it much less clear what the operations are between those parameters

https://sourceware.org/binutils/docs/as/i386_002dMemory.html#i386_002dMemory

Intel vs. AT&T Syntax 7

- Examples:
  - call qword ptr [rbx+rsi*4-0xe8]
  - callq *-0xe8(%rbx,%rsi,4)

  - mov rax, qword ptr [rbp+0x8]
  - movq 0x8(%rbp), %rax

  - lea rax, [rbx-0xe8]
  - leaq -0xe8(%rbx), %rax

Intel vs. AT&T Syntax 8

```
endbr64
sub    $0x1000,%rsp
orq    $0x0,(%rsp)
sub    $0x1000,%rsp
orq    $0x0,(%rsp)
sub    $0x28,%rsp
movl   $0xff,0x1010(%rsp)
mov    0x8(%rsi),%rdi
mov    $0xa,%edx
mov    $0x0,%esi
callq  0x555555555070 <strtol@plt>
movslq %eax,%rdx
lea    0x1010(%rsp),%rsi
mov    %rsp,%rdi
mov    $0x1004,%ecx
callq  0x555555555060 <__memcpy_chk@plt>
movabs $0xace0fba5e,%rax
add    $0x2028,%rsp
retq
```

```
endbr64
sub    rsp,0x1000
or     QWORD PTR [rsp],0x0
sub    rsp,0x1000
or     QWORD PTR [rsp],0x0
sub    rsp,0x28
mov    DWORD PTR [rsp+0x1010],0xff
mov    rdi,QWORD PTR [rsi+0x8]
mov    edx,0xa
mov    esi,0x0
call   0x555555555070 <strtol@plt>
movsxd rdx,eax
lea    rsi,[rsp+0x1010]
mov    rdi,rsp
mov    ecx,0x1004
call   0x555555555060 <__memcpy_chk@plt>
movabs rax,0xace0fba5e
add    rsp,0x2028
ret
```

Intel vs. AT&T Syntax 9

AT&T

```
movl    $0xff,0x1010(%rsp)
mov     0x8(%rsi),%rdi
mov     $0xa,%edx
mov     $0x0,%esi
```

Intel

```
mov     DWORD PTR [rsp+0x1010],0xff
mov     rdi,QWORD PTR [rsi+0x8]
mov     edx,0xa
mov     esi,0x0
```

Shouldn't this be movq?

How do I know it's moving a qword and not a byte?

Short answer is that *you don't yet*. ¯_(ツ)_/¯ (Another reason I prefer Intel over AT&T syntax)

But once you learn to RTFM later, you'll at least be able to figure it out